

Desafio Técnico - Sistema de Biblioteca

Este projeto tem como objetivo avaliar suas habilidades técnicas em **C#**, boas práticas de desenvolvimento e capacidade de resolução de problemas em um contexto real.

O sistema base simula o controle de uma **biblioteca**, permitindo o **cadastro de usuários, livros e empréstimos**, além do controle de devoluções e cálculo de multas.

Os desafios estão listados a seguir e possuem diferentes níveis de complexidade.

Nem todos os desafios são obrigatórios, mas recomendamos que você implemente **o máximo que puder**, priorizando qualidade de código, clareza nas regras de negócio e boas práticas de arquitetura.

Desafios

1. FEATURE: Validar CPF do usuário

- No cadastro de usuário, validar se o **CPF já existe** antes de cadastrar.
- Caso o CPF já esteja cadastrado, retornar:

```
{  
    "sucesso": false,  
    "conteudo": null,  
    "mensagem_erro": "Usuário com este CPF já está cadastrado."  
}
```

2. FEATURE: Validar ISBN do livro

- O campo ISBN deve conter **exatamente 13 dígitos numéricos**.
- Caso o formato esteja incorreto, retornar erro apropriado.

3. BUG: Gerando multa mesmo quando o empréstimo é devolvido no prazo

- Corrigir o cálculo da multa: ela deve ser gerada **somente** se a devolução ocorrer **após a data_previsita_devolucao**.

4. BUG: Permite emprestar um livro que já está emprestado

- Antes de registrar um novo empréstimo, verificar se o **livro já está emprestado e ainda não foi devolvido**.
- Caso esteja, retornar:

```
{  
    "sucesso": false,  
    "conteudo": null,  
    "mensagem_erro": "Este livro já está emprestado e ainda não foi devolvido."  
}
```

5. FEATURE: Listar todos os livros

- Criar endpoint GET /livro/listar
- Deve retornar a lista completa de livros cadastrados:

```
{  
    "sucesso": true,  
    "conteudo": [  
        {  
            "id": 1,  
            "titulo": "O Senhor dos Anéis",  
            "autor": "J.R.R. Tolkien",  
            "isbn": "S788533c1337S"  
        }  
    ],  
    "mensagem_erro": null  
}
```

6. FEATURE: Impedir novo empréstimo se o usuário tiver atraso

- Validar se o usuário possui algum empréstimo **em atraso** (ainda não devolvido e `data_prevista_devolucao < DateTime.Now`).
 - Caso possua, o sistema deve **impedir novo empréstimo** e retornar a mensagem:
"Usuário com empréstimo em atraso não pode realizar novo empréstimo."
 - Criar uma **marcação no banco de dados** para indicar se o usuário possui atraso ativo.
-

7. FEATURE: Ajustar regra de multa

- Implementar nova lógica para cálculo de multa:
 - Até **3 dias de atraso** → R\$ 2,00 por dia
 - A partir do **4º dia** → R\$ 3,50 por dia
 - **Limite máximo:** R\$ 50,00
 - Exemplo:
 - 2 dias → R\$ 4,00
 - 5 dias → $(3 \times 2,00) + (2 \times 3,50) = \text{R\$ } 13,00$
 - 20 dias → R\$ 50,00 (limitado)
-

8. FEATURE: Implementar autenticação (Bearer / API Token)

- Implementar autenticação para proteger endpoints sensíveis (ex: cadastro de livro, empréstimos, devoluções).
 - A autenticação pode ser:
 - **Bearer Token (JWT)**
 - ou **API Token** simples.
 - Retornar erro 401 Unauthorized quando o token for inválido.
-

Entrega do Desafio

1. Crie um **repositório público no GitHub** com a sua solução.
2. Inclua um arquivo **README.md** contendo:
 - Instruções detalhadas de como **executar o projeto**, especialmente se houver **containerização (Dockerfile ou docker-compose)**.
 - Uma **lista dos desafios realizados**, marcando claramente quais foram implementados.
3. Após finalizar, **compartilhe o link do repositório** conosco para avaliação.